Laboratório 06 — o agente de linha de base

D2 — Fundamentos de Agentes Inteligentes · Encontro 4, bloco 2 (manhã) Pacote: `D2-06-agente-baseline.zip` · 70 minutos, contando a montagem do conjunto

O que você já tem, e o que falta

Você já tem	O que falta
O `agent-spec.md` com a medida de desempenho definida (lab-01 e AO1)	O agente: as regras do SEU domínio no esqueleto
O esqueleto rodando sobre a reserva: `python medir.py` imprime métrica, valor e conjunto	O conjunto de 30 casos com desfecho conhecido — o seu, ou a reserva
O conjunto de reserva, 30 casos rotulados, no pacote (`COMO-USAR-A-RESERVA.pdf`)	A medição rodando sobre os 30
A estrutura do slide 22: `decidir(registro, estado)`, com estado interno opcional	Duas frases de análise: onde o agente erra, e por quê

O número que sai daqui é a linha de base contra a qual o agente de D10 vai ser comparado. Comece PELOS 30 casos — sem eles não há o que medir — e anote o número assim que sair, antes de tentar melhorar: o primeiro resultado, mesmo ruim, É a linha de base.

Os dois caminhos

- **Padrão:** o esqueleto chega com a estrutura e as marcações de tarefa — o trabalho é preencher as condições com os limiares do seu domínio e montar o conjunto.
- **Desafio:** escreva o agente do zero — função de decisão, estado interno, e a mesma medição. O `solucao.py` é o gabarito comentado, com a análise dos erros como modelo.

Trocar de caminho é livre, nos dois sentidos e sem justificar. Quem estiver no padrão e travar mais de dez minutos numa regra simplifica para UMA condição e segue: agente com uma regra e trinta casos medidos vale a nota; agente elegante sem medição, não.

Os passos

1. Extraia o `D2-06` na raiz do repositório.
2. Monte o conjunto de 30 casos com desfecho conhecido — 30 registros do seu conjunto de dados com a coluna do rótulo, ou a reserva do pacote. Inclua casos fáceis, casos difíceis e pelo menos dois atípicos: conjunto só de casos fáceis produz um número bonito e inútil.
3. Preencha as regras do seu domínio no `esqueleto.py` — os TODOs 1 e 2 marcam a condição reativa e a condição com estado interno. Quem caracterizou o ambiente como totalmente observável (lab-02) pode APAGAR a segunda condição — e registra a escolha na análise, porque o tipo de agente é parte do que a rubrica avalia.
4. Rode a medição:

```

python medir.py

```

A saída traz as três informações do slide 28 — a métrica, o valor e sobre que conjunto — mais a lista de onde erra.

1. Anote a métrica primária do seu PEAS, com o valor e o conjunto.
2. Escreva duas frases: onde o agente erra, e por quê. A lista do `medir.py` é a matéria-prima — todo agente de regras erra de forma sistemática, não aleatória, e o padrão do erro é o que diz que tipo de agente o problema realmente exige (é o item 5 da entrega final).
3. Rode `python -m pytest teste.py -q`. Quando o baseline estiver pronto, copie as funções para `src/<projeto>/baseline.py` — o caminho que a entrega final cobra.

Critério de aceite

`baseline.py` roda sobre 30 casos e devolve a métrica primária — o mesmo critério do slide 26, literal.

Quanto tempo isto deve levar

Setenta minutos, contando a montagem do conjunto. A ordem que funciona: casos primeiro (uns 20 minutos se o dado é seu; zero se for a reserva), depois as regras, depois a medição e as duas frases de análise.

Se travar

1. **O conjunto não tem desfecho?** Rotule os trinta à mão — é uma tarde. Ou use a reserva hoje e troque depois: a troca é uma linha (`CAMINHO_DOS_CASOS` no topo do esqueleto), e o `COMO-USAR-A-RESERVA.pdf` conduz a troca inteira.
2. **Erro de tipo ao ler o registro?** Confira o `carregar_casos` do esqueleto: ele converte idade, temperatura, vibração e o desfecho — coluna escrita diferente no seu CSV é a causa mais comum.
3. **Quer escrever a regra perfeita?** Não tente — duas regras já são um agente, e a vontade de cobrir todos os casos com regras é exatamente o erro que produziu o segundo inverno da IA. O que a rubrica premia é o número medido e a análise honesta de onde ele erra.
